FORMACIÓN · 12 HORAS · 4 SESIONES

# Gestión de proyectos de red

## con enfoque IaC, Agile y NetDevOps

SESIÓN 1

**El cambio de paradigma**

```
$ git init red-como-codigo
```

Módulos 1 y 2 · Infraestructura como Código · Agile aplicado a trabajos de red

LA HISTORIA · 15 MIN

# De la gestión clásica al trabajo vivo

### El mundo para el que se inventó

- Proyectos cerrados: alcance, fases, plan detallado
- Control por hitos y entrega final
- El plan es la verdad; desviarse es fracasar
- Funciona cuando el trabajo es predecible

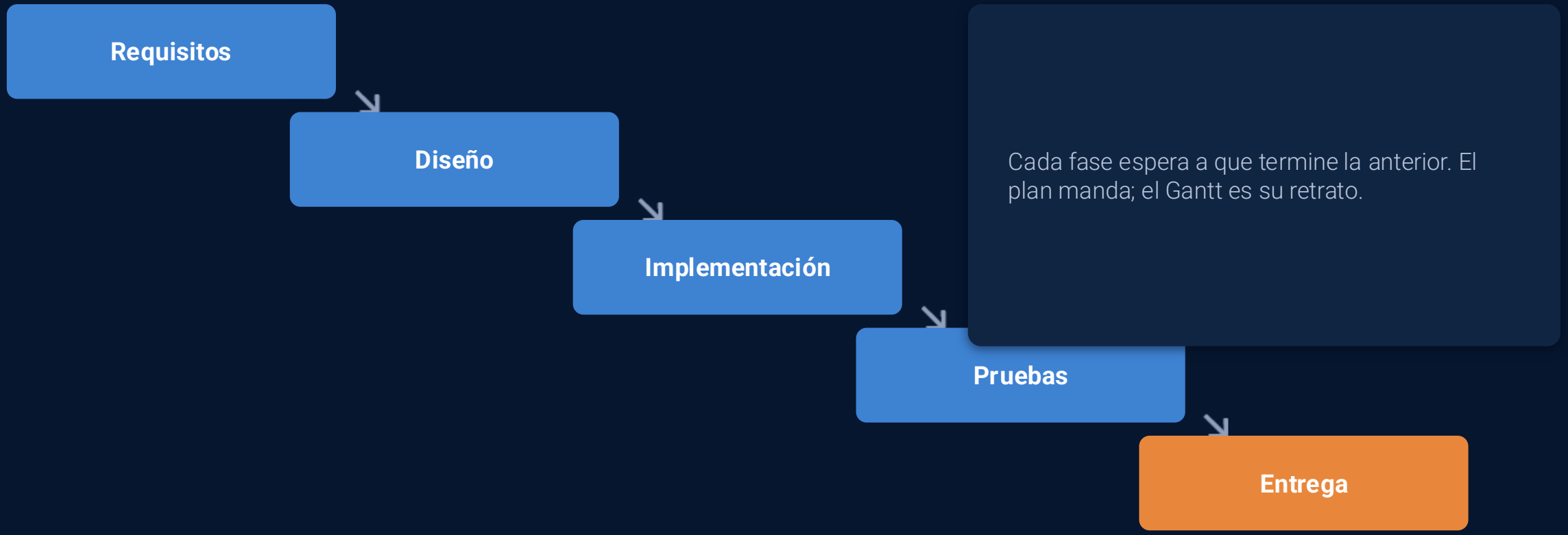
### El mundo real de un servicio de red

- Cambios continuos, incidencias y urgencias que rompen el plan
- Dependencias externas: operador, compras, el propio cliente
- El cliente pide visibilidad hoy, no en la entrega final
- El servicio nunca "termina": convive proyecto y operación

*La gestión clásica no está "mal": está diseñada para otro mundo. Todo lo que viene a continuación nace de esta fricción.*

LA HISTORIA

# El modelo en cascada, dibujado




*El aprendizaje y el riesgo llegan al final — cuando corregir es más caro. En un mundo predecible, funciona; en un servicio vivo, cruje.*

LA HISTORIA · 15 MIN

# Dos respuestas: Agile y DevOps



## Agile (2001)

- No nace para hacer reuniones raras
- Nace para hacer visible el trabajo y priorizarlo
- Entregar en incrementos: valor y aprendizaje pronto
- Adaptarse al cambio en vez de negarlo



## DevOps (~2009)

- Agile mejoró la construcción… y chocó con la operación
- Dos mundos separados: "los que hacen" y "los que operan"
- DevOps los une: flujo, feedback y responsabilidad de servicio
- El objetivo no es la herramienta: es el servicio que funciona

*¿Os suena el choque construcción vs operación? En un servicio de red es proyecto vs operación diaria — lo trabajaremos en la Sesión 2.*

LA HISTORIA · AGILE EN LA PRÁCTICA

# Iteraciones: qué pasa cuando algo no llega

Backlog priorizado por valor: R1, R2, R3, R4, R5… Llega el 10 de agosto y R3 no está terminado:

### Con hitos (clásico)

- Suenan las alarmas y se buscan responsables
- Se replanifica el hito: nueva fecha, 15 de agosto
- El cliente no tiene nada hasta la entrega final
- Vamos con retraso — y sin información nueva

### Con iteraciones (Agile)

- Se entrega R1 y R2: el cliente ya recibe valor
- R3 pasa a la siguiente iteración
- El cliente da feedback sobre lo que ya tiene
- Seguimos con retraso — pero con control y con información

*El retraso existe en los dos mundos. La diferencia es qué tienes entre las manos cuando ocurre.*

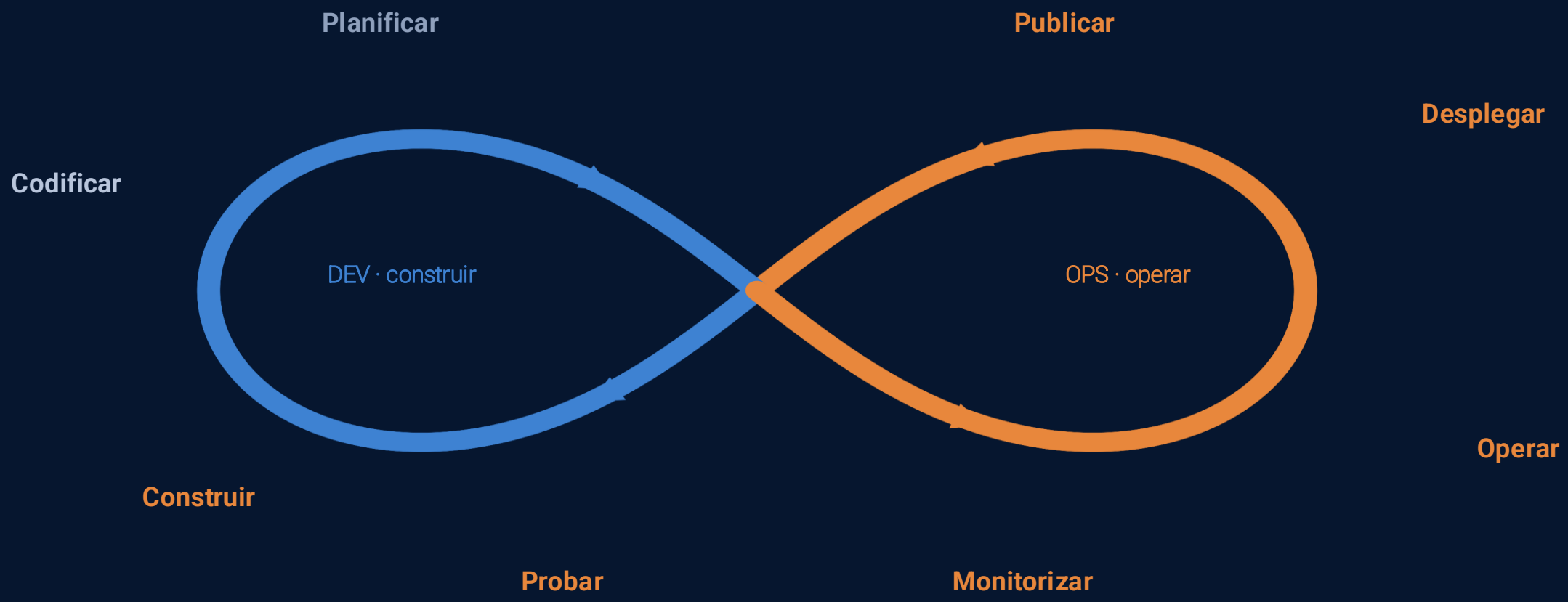
LA HISTORIA · EL PRECIO DE AGILE

# La cuenta que no sale (y su única salida)

1. **Entregas frecuentes** — Cada pocas semanas, no una vez al final
2. **Las pruebas se disparan** — Cada entrega exige probar lo nuevo… y re-probar lo anterior
3. **Los despliegues se multiplican** — De instalar una vez a desplegar continuamente
4. **No hay dinero, tiempo ni gente** — Hacer todo eso a mano es sencillamente inviable
5. **Automatizar** — La única salida realista. Y de ahí nace la cultura DevOps

LA HISTORIA

# El ciclo DevOps: lo que se automatiza y monitoriza




*En naranja, lo hoy altamente automatizable; en gris, lo que sigue siendo trabajo de personas. DevOps es recorrer este bucle sin fin — cada vuelta, una entrega.*

LA HISTORIA · 15 MIN

# El mapa: de la gestión clásica a NetDevOps

**Clásica** → **Agile** → **DevOps** → **IaC** → **NetDevOps**

| Clásica | Agile | DevOps | IaC | NetDevOps |
|---|---|---|---|---|
| Planificación, fases, control por hitos | Visibilidad, priorización, incrementos, adaptación | Flujo y feedback entre construcción y operación | La infraestructura descrita como software: versionable y trazable | Todo lo anterior aplicado al servicio de red |

La idea que une el curso: si la red puede describirse como software, el servicio puede gestionarse con trazabilidad, evidencias, revisión y automatización progresiva. Eso es lo que veremos ahora con IaC.

AUTOMATIZAR · 30 MIN

# ¿Qué es automatizar? La lavadora y la persiana

## Automatizar una TAREA

La lavadora automatiza el lavado. Pero el proceso sigue teniendo tareas humanas: cargar la ropa, poner detergente, elegir programa, pulsar el botón.

La persiana con motor y botón: tarea automatizada. Falta decidir cuándo y pulsar.

## Automatizar el PROCESO

El asistente de hogar decide y pulsa: "sube a las 9:00, baja a las 21:00". Ahora sí: el proceso completo funciona sin humano.

Regla de oro: no puedes automatizar un proceso hasta tener automatizadas sus tareas.

Automatizar no es barato. Compensa cuando el trabajo es repetitivo (se hará muchas veces), crítico (un error sale caro) o exige trazabilidad (a mano no queda rastro). Tres criterios que en un servicio de red se cumplen casi siempre.

MÓDULO 1

# Infraestructura como Código aplicada a redes

```
$ ansible-playbook cambio_vlan.yml --check
```

MÓDULO 1

# ¿Qué significa tratar la red como software?

No es programar. Es aplicar a la red las prácticas que el software resolvió hace años:



### La configuración es un artefacto

Deja de vivir solo dentro del equipo: se guarda, se compara y se revisa como un fichero.



### Todo cambio deja historia

Quién cambió qué, cuándo y por qué. Sin arqueología ni memoria tribal.



### El estado deseado es explícito

Lo que "debería estar configurado" está escrito, no en la cabeza de alguien.



### Los cambios son repetibles

Un mismo cambio se aplica igual en 1 o en 50 equipos, con revisión previa.

*Durante años, operaciones exigió a desarrollo entornos separados, versiones y dependencias claras… sin aplicárselo a su propia infraestructura. IaC es saldar esa deuda.*

MÓDULO 1

# Del cambio manual al cambio versionado y trazable

### HOY: cambio manual

- Un técnico entra al equipo y cambia la config
- El motivo del cambio vive en un correo (o en nadie)
- Si algo falla, al cliente se le responde "lo estamos mirando"
- Volver atrás depende de la memoria del técnico que lo hizo
- El conocimiento se concentra en personas concretas

### MAÑANA: cambio versionado

- El cambio se propone como modificación de un fichero
- El motivo queda escrito junto al cambio (tarea + commit)
- Alguien lo revisa y aprueba antes de aplicarlo
- Volver atrás = volver a la versión anterior
- La historia completa está disponible: también para responder al cliente

IAC

# Versiones de la aplicación, versiones de la infraestructura

La red por sí sola no vale nada: existe para que los servicios funcionen. Por eso sus versiones van de la mano:

**`App v1.0.0 → Infra v1.0.0`**

2 servidores de aplicación, balanceador, BBDD, dos subredes, sus IPs y sus reglas de firewall.

**`App v2.0.0 → Infra v1.1.0`**

La app añade una caché (Redis): la infra necesita un servidor más y nuevas reglas de firewall. Nueva versión.



El día del rollback: la v2.0.0 falla en producción y hay que volver a la v1.0.0 de la app… lo que exige devolver la infra a su v1.0.0. Sin versionado, ese "volver atrás" es arqueología nocturna; con él, es ir a una versión conocida.

*Esto es la gestión de configuraciones: saber qué versión de la infra hay en producción y qué versión necesita cada servicio.*

MÓDULO 1

# Artefactos técnicos del servicio, gestionados como código



## Configuraciones

Cómo está montada la red de cada centro, con historia

`lugo_sw01.cfg`



## Inventarios

Qué equipos existen, dónde están, qué rol tienen

`inventario_sedes.yml`



## Plantillas

Configuraciones tipo: "así se da de alta una sede"

`plantilla_sede.j2`



## Políticas

Reglas que deben cumplirse: ACLs, versiones, estándares

`politica_acl.yml`

*No los gestionaréis vosotros: el valor es que existan y estén versionados — porque eso da trazabilidad, evidencias y respuestas al cliente.*

MÓDULO 1

# Git: el registro civil de los cambios de red

Git guarda versiones de ficheros. Para gestión, lo importante es lo que eso habilita:



### Propuesta

Un cambio se propone como nueva versión del fichero



### Revisión

Otra persona ve exactamente qué líneas cambian



### Aprobación

El visto bueno queda registrado y con fecha



### Historial

Cada versión, autor y motivo, consultable siempre

Un ejemplo del historial que genera:

```
a3f21c  2026-07-02  m.lopez   RED-341: ampliar VLAN voz centro Lugo (aprobado: j.ruiz)
```

MÓDULO 1

# Automatización progresiva (y sus riesgos)

### 1 Documentar

Configs e inventarios versionados. Solo lectura: nadie toca nada.

### 2 Validar

Comprobaciones automáticas: ¿la red real coincide con lo escrito?

### 3 Asistir

Las herramientas generan el cambio; una persona lo aplica.

### 4 Automatizar

El cambio aprobado se aplica solo, con control y vuelta atrás.



Riesgo real: automatizar sin gobierno multiplica los errores a la velocidad de la máquina. Primero proceso y aprobaciones; después, automatización.

MÓDULO 1

# ¿Qué gana la gestión de proyectos con IaC?

### Trazabilidad

Cada cambio enlaza tarea, motivo, autor y aprobación. Auditorías sin arqueología.

### Control de cambios

El cambio se revisa antes de tocar la red, no se reconstruye después del incidente.

### Coordinación

Todo el equipo ve el mismo estado deseado; menos dependencia de personas clave.

### Visibilidad

El avance del proyecto se ve en tareas y versiones, no en promesas verbales.

### Vuelta atrás

Revertir un cambio es volver a una versión conocida, con menos improvisación.

### Medición

Cambios, incidencias y tiempos quedan registrados: base para los cuadros de mando.

RECAPITULACIÓN

# Cinco ideas para llevarse hoy

1. La gestión clásica no está mal: está diseñada para trabajo predecible. El servicio de red vive en otro mundo: cambios, incidencias y un cliente que pide visibilidad hoy.

2. Agile = entregar en incrementos para obtener feedback pronto. Su precio: las pruebas y los despliegues se multiplican — y la única salida realista es automatizar.

3. Automatizar es caro: compensa cuando el trabajo es repetitivo, crítico o exige trazabilidad. Primero se automatizan tareas; solo después, procesos.

4. IaC = tratar la infraestructura como software: versionada, con historia (qué, quién, cuándo, por qué) y con el programa como fuente de verdad.

5. El trabajo del equipo técnico se desplaza: de operar la red a crear los programas que la operan. Para el servicio, eso significa evidencias, repetibilidad y mejores respuestas al cliente.

MAÑANA

# La palabra pendiente: NetDevOps

**NetDevOps**

Qué es (y qué no es) aplicar todo lo de hoy al servicio de red — y de dónde sale el término

**Gestión del servicio**

Asuntos vivos, priorización, bloqueos y visibilidad — con dos dinámicas sobre vuestra realidad

*Traed en la cabeza una petición real y reciente de un centro del cliente: trabajaremos sobre ella.*